UNIVERSIDAD COOPERATIVA DE COLOMBIA

Facultad de Ingeniería · Ingeniería de Software

Última Dosis

Motor de juego 2D por turnos con pathfinding

Informe técnico del proyecto integrador · Temática A

Curso	Estructuras de Datos · ID 706132
Estudiante	Juan José Correa
Modalidad	Individual
Repositorio	github.com/correajuanjose200-coder/proyecto-ed-correajuanjose200-coder
Etiqueta	entrega-final
Fecha	11 de noviembre de 2026

1. Descripción del sistema

Última Dosis es un juego por turnos que se juega en la consola. Pedro, el protagonista, debe cruzar una ciudad infectada hasta el aeropuerto (la celda **E**) mientras dos zombis lo persiguen. El mapa es una rejilla de 20 x 20 celdas, con obstáculos que ocupan como máximo el 25 % de ellas. En el camino hay comida (**C**) y agua (**A**) que se recogen al pisarlas y se guardan en un inventario de hasta cinco objetos.

En cada turno Pedro puede moverse una casilla (w, a, s, d), esperar (x), usar un objeto (u N), soltar un objeto (g N), deshacer (z) o rehacer (r). Después de cada acción válida, cada zombi calcula una ruta hacia Pedro y avanza una casilla por ella. Si un zombi queda adyacente a Pedro, lo ataca y le quita 25 puntos de salud. La partida empieza con salud 100; la comida suma 20 y el agua suma 10, sin pasar de 100. Hay victoria al llegar a la celda E y derrota si la salud llega a 0; si ambas cosas ocurren en el mismo turno, la victoria tiene prioridad (CA-28).

El proyecto no se centra en los gráficos sino en las decisiones de estructuras de datos que hay debajo: qué estructura resuelve cada capacidad del dominio (deshacer, inventario, eventos, rutas, mapa), con qué complejidad y frente a qué alternativa. Todas las estructuras se implementaron desde cero, sin usar `list`, `dict`, `set`, `deque` ni `heapq` como sustituto de una estructura declarada.

2. Estado de la entrega

Este informe describe lo que realmente está implementado y probado en la etiqueta `entrega-final`. Se declara con honestidad lo que no se alcanzó a construir.

Componente	Estado	Módulo
Pila	Implementada y probada	estructuras/pila.py
Cola	Implementada y probada	estructuras/cola.py
Inventario (lista enlazada)	Implementado y probado	estructuras/inventario.py
Historial (deshacer y rehacer)	Implementado y probado	dominio/historial.py, dominio/accion.py
Mapa (matriz densa, generación por semilla)	Implementado y probado	dominio/mapa.py
Rutas (grafo implícito + BFS)	Implementadas y probadas	dominio/rutas.py
Juego y consola	Implementados; pruebas de integración parciales	dominio/juego.py, main.py
AVL (índice de entidades)	No implementado	-
Montículo (ranking)	No implementado	-
Cola de turnos propia	No implementada (orden fijo)	-
Antídoto (CA-05) y aturdimiento (CA-22)	No implementados	-

La suite tiene 126 pruebas con pytest y pasa completa. Las funcionalidades F-08 (buscar entidad y ranking, criterios CA-30 a CA-34) quedaron sin cubrir porque dependían del AVL y del montículo; esto está registrado como cambio de prioridad en `specs/00-producto/moscow.md` versión 1.1.

3. Arquitectura y módulos

El sistema se divide en tres capas. La capa de **estructuras** contiene las estructuras de datos genéricas, sin conocimiento del juego. La capa de **dominio** contiene las reglas del juego y usa las estructuras. La capa de **consola** (main.py) solo lee comandos y dibuja. Cada estructura vive en su propio módulo, con docstrings que declaran la complejidad de cada operación pública.

Capa	Módulo	Responsabilidad	Usa
Consola	main.py	Bucle de comandos, dibujo del mapa y mensajes	Juego
Dominio	juego.py	Reglas de turno, zombis, salud, victoria y derrota; coordina todo	Mapa, Rutas, Historial, Inventario, Cola
Dominio	rutas.py	Camino más corto (BFS) y primer paso	Mapa, Cola
Dominio	historial.py / accion.py	Registro de acciones, deshacer y rehacer	Pila
Dominio	mapa.py	Rejilla, generación por semilla, dibujo	-
Estructuras	pila.py, cola.py	LIFO y FIFO sobre nodos enlazados	Nodo
Estructuras	inventario.py	Lista enlazada con puntero al último nodo	Nodo
Estructuras	nodo.py	Nodo enlazado simple	-

Flujo de un turno. Cuando Pedro ejecuta una acción válida, `Juego` guarda el estado de antes (posición y salud de Pedro, posición de cada zombi, contador de turnos), aplica la acción, mueve a los zombis con BFS, y registra una `Accion` con el estado de antes y de después en el `Historial`. Los sucesos (aparición, recolección, colisión, deshacer y rehacer) se encolan como eventos en una `Cola` y la consola los muestra una sola vez y en orden de llegada.

Por qué la consola no abre el bucle al importarse. La función `main()` solo imprime el encabezado y el bucle interactivo corre únicamente con `python main.py`. Así la prueba de arranque de pytest no se queda esperando una entrada.

4. Aplicación de Spec-Driven Development

El flujo de trabajo fue: historia de usuario, priorización MoSCoW, spec.md (contrato con criterios CA), plan.md (estructura y complejidad), tasks.md y, por último, pruebas y código. Cada componente tiene su carpeta en `specs/` y cada prueba nombra en su docstring el criterio de aceptación que verifica; la tabla criterio-prueba está en `specs/trazabilidad.md`.

Entrega 1 (14 de octubre): constitución, historias de usuario con sus criterios (CA-01 a CA-36), MoSCoW, tabla de estructuras y el componente de historial completo. Entrega 2: pila, historial, acción y mapa. Entrega final: cola, inventario, rutas, juego y documentos de cierre. En el historial de Git se ve el orden spec, prueba y código para cada componente, por ejemplo `spec(rutas)`, luego `test(rutas)` y luego `feat(rutas)`.

Cambios de especificación y su motivo.

Documento	Cambio	Motivo
spec del historial v1.1	CA-H2 se hizo coherente con las reglas del historial	Se detectó una contradicción al escribir las pruebas de deshacer y rehacer
plan del historial v1.1	Se declaran las tuplas como soporte de Accion	Regla 4 del proyecto: todo soporte interno debe justificarse
moscow.md v1.1	AVL y montículo pasan de Must a Won't; CA-05 y CA-22 pendientes	Falta de tiempo; se priorizó un juego jugable con pruebas
estructuras.md v1.1	Se registra que no hay cola de turnos propia ni AVL ni montículo	Coherencia con lo implementado
99-sistema/spec.md v1.1	Componentes reales, reglas y lista de lo no implementado	Cierre de la entrega final

Durante la integración se detectó además que el primer borrador de `juego.py` usaba valores distintos a los de la especificación (salud 3 en lugar de 100, comida +2 en lugar de +20, y la derrota evaluada antes que la victoria). Se corrigió el código para que coincida con la spec (CA-05, CA-10, CA-22, CA-28) y se ajustaron las pruebas, en lugar de cambiar la especificación.

5. Las estructuras de datos

Para cada estructura se indica qué es, cómo se implementó, dónde se usa, su complejidad y la alternativa descartada.

5.1 Pila (dos pilas para deshacer y rehacer)

Qué es. Una estructura LIFO: el último elemento que entra es el primero que sale. **Implementación.** Nodos enlazados (`Nodo`) con un puntero a la cima; no usa `list`. Operaciones: apilar, desapilar, cima, tamano, esta_vacia y vaciar. Desapilar o leer la cima de una pila vacía lanza `PilaVaciaError`.

Dónde se usa. `Historial` mantiene dos pilas: *hechas* (acciones que se pueden deshacer) y *deshechas* (acciones que se pueden rehacer). Registrar una acción nueva apila en hechas y vacía deshechas, lo que cumple CA-17. Deshacer desapila de hechas y apila en deshechas; rehacer hace lo contrario. Si no hay nada que deshacer o rehacer, el historial devuelve None sin cambiar el estado (CA-18).

Complejidad. Apilar, desapilar, cima, tamano y esta_vacia son O(1). Vaciar recorre los nodos, O(n), según la implementación. **Alternativa descartada.** Una lista doble con cursor también da O(1), pero obliga a mantener el cursor y a cortar la lista cada vez que se registra una acción nueva; es más código para el mismo resultado.

5.2 Cola (eventos y frontera del BFS)

Qué es. Una estructura FIFO: el primero que entra es el primero que sale. **Implementación.** Nodos enlazados con punteros al frente y al final, de modo que encolar y desencolar no recorren la estructura. Desencolar o consultar el frente de una cola vacía lanza `ColaVaciaError`.

Dónde se usa. En dos lugares con la misma necesidad de orden de llegada. Primero, los eventos del juego salen en el mismo orden en que ocurrieron y cada uno se muestra una sola vez (CA-23 y CA-25). Segundo, es la frontera del recorrido en anchura de `rutas.py`. **Complejidad.** Encolar, desencolar, frente, tamano y esta_vacia son O(1). **Alternativa descartada.** Una cola de prioridad: todos los eventos tienen la misma urgencia, así que la prioridad solo añadiría O(log n) sin beneficio.

5.3 Inventario (lista enlazada simple)

Qué es. Una secuencia ordenada de objetos con capacidad máxima de cinco. **Implementación.** Lista enlazada simple con referencia al último nodo. Agregar inserta al final; insertar y quitar trabajan por posición (desde 0) y listar recorre en orden. Agregar con el inventario lleno lanza `InventarioLlenoError`, y una posición inexistente lanza `PosicionInventarioError`.

Dónde se usa. Recoger un objeto lo agrega al final y conserva el orden de recogida (CA-09, CA-13). Usar y soltar lo quitan por posición. Como el inventario no conoce los efectos de los objetos, deshacer puede reinsertar un objeto en su posición original: `Accion.objeto` guarda el símbolo y la posición. Cuando el inventario está lleno, el objeto se queda en el mapa y se genera un evento (CA-12).

Complejidad. Agregar es O(1) gracias al puntero al último nodo. Insertar, quitar y listar son O(n), con n como máximo 5, por lo que el costo es despreciable. **Alternativa descartada.** Un arreglo de capacidad 5: con ese tamaño el costo sería igual, pero quitar del medio obliga a desplazar elementos

y a manejar posiciones vacías.

5.4 Mapa (matriz densa)

Qué es y cómo se implementó. Una rejilla de filas por columnas con los símbolos libre (.), obstáculo (#), meta (E), comida (C) y agua (A). Las operaciones obtener, poner, dentro y es_transitable son accesos directos por índice. La generación recibe una semilla, por lo que la misma semilla produce el mismo mapa; coloca obstáculos sin superar el 25 % y garantiza un corredor transitable entre el inicio y la meta (CA-01 y CA-02).

Complejidad. Consultar o cambiar una celda es O(1); contar, dibujar y calcular la proporción de obstáculos son O(F x C). **Soporte interno.** La matriz usa `list` solo como contenedor de tamaño fijo; todas las operaciones están implementadas en `Mapa`. **Alternativa descartada.** Una matriz dispersa solo ahorraría memoria si casi todo estuviera vacío; con 400 celdas y hasta 25 % de obstáculos, añadiría una búsqueda en cada consulta.

5.5 Grafo implícito y búsqueda en anchura (BFS)

Qué es. Cada celda transitable es un vértice y cada par de celdas vecinas transitables es una arista. El grafo no se construye: los vecinos (arriba, abajo, izquierda y derecha, en orden fijo) se calculan al vuelo con `dentro` y `es_transitable`. Esto ahorra memoria y evita mantener aristas cuando el mapa cambia.

Implementación. `buscar_ruta(mapa, origen, destino)` usa la `Cola` propia como frontera y una matriz de padres. La matriz cumple dos funciones: una celda con valor None aún no se ha descubierto, y el valor guardado es la celda desde la que se llegó. Al descubrir el destino se camina hacia atrás por los padres, contando pasos; la última celda antes del origen es el primer paso. La función devuelve un objeto con distancia, siguiente y exploradas, devuelve None si no hay camino, lanza `PosicionInvalidaError` si un extremo está fuera del mapa y no modifica el mapa (CA-R1 a CA-R9, CA-19, CA-20, CA-21).

Por qué da el camino más corto. La cola es FIFO, así que el recorrido avanza por capas: todas las celdas a distancia k se procesan antes que las de distancia k + 1. La primera vez que se descubre una celda es, por tanto, por un camino de longitud mínima. El orden fijo de vecinos hace que el resultado sea determinista incluso con empates.

Complejidad. O(V + E), que en la rejilla equivale a O(F x C) por zombi y por turno; con dos zombis, un turno cuesta O(Z x F x C). **Soporte interno.** La matriz de padres usa `list` y se declara en el plan. **Alternativa descartada.** A*: exploraría menos celdas con una heurística admisible (la distancia de Manhattan lo es en una rejilla de cuatro vecinos), pero con unas 400 celdas la diferencia no se nota y BFS es más simple de probar y de explicar. A* queda como mejora; el campo `exploradas` ya permite medir la comparación.

6. Análisis de complejidad

Módulo	Operación	Complejidad
pila	apilar, desapilar, cima, tamano, esta_vacia	O(1)
pila / cola	vaciar	O(n)
cola	encolar, desencolar, frente, tamano, esta_vacia	O(1)
inventario	agregar	O(1)
inventario	insertar, quitar, listar	O(n), n <= 5
historial	registrar, deshacer, rehacer	O(1)
mapa	obtener, poner, dentro, es_transitable	O(1)
mapa	contar, dibujar, proporcion_obstaculos	O(F x C)
rutas	buscar_ruta (BFS)	O(V + E) = O(F x C)
juego	un turno	O(Z x F x C)
juego	deshacer, rehacer	O(1) en el historial, más O(n) del inventario si hay objeto

La operación que domina el costo del juego es la búsqueda de rutas: un BFS por zombi en cada turno. Con un mapa de 20 x 20 y dos zombis son como máximo unos 800 vértices visitados por turno, un costo que no afecta la jugabilidad. Todo lo demás (historial, eventos, inventario) es constante o depende de n como máximo 5.

7. Pruebas y trazabilidad

La suite tiene 126 pruebas con pytest y pasa completa. Hay pruebas por estructura (pila, cola, inventario, historial, acción, mapa, generación de mapa, rutas) y pruebas de integración en `tests/test_juego.py`. Las pruebas cubren los extremos: estructuras vacías, un solo elemento, posiciones inexistentes, mapas de una celda, destinos encerrados y orígenes sobre obstáculos. Cada prueba indica en su docstring el criterio de aceptación que verifica.

Las pruebas de integración verifican, entre otros, CA-15 (deshacer una recogida devuelve el objeto al mapa y lo quita del inventario), CA-16 (rehacer), CA-17 (una acción nueva vacía lo rehacible), CA-07 (los obstáculos bloquean y no gastan turno), CA-10 (usar comida suma 20), CA-23 (eventos en orden), y CA-26 y CA-27 (victoria y derrota). La trazabilidad completa está en `specs/trazabilidad.md`.

Cobertura incompleta. Varios criterios están implementados pero sin una prueba de integración dedicada (CA-03, CA-04, CA-06, CA-08, CA-09, CA-11 a CA-14, CA-18, CA-24, CA-28, CA-29, CA-35 y CA-36), y el criterio CA-22 está solo parcialmente cubierto: el bloqueo y el daño existen, pero no el aturdimiento. La trazabilidad lo declara como pendiente.

8. Dificultades

La principal dificultad fue la gestión del tiempo: la integración del juego, las estructuras de cola, inventario y rutas, y los documentos de cierre se concentraron en los últimos días del plazo. Por eso

no se alcanzaron a construir el AVL ni el montículo, y la cola de turnos se resolvió con un orden fijo.

También hubo dificultades técnicas. Al pegar archivos largos en el editor del entorno, algunos archivos quedaron vacíos y las pruebas fallaron con errores de importación; se resolvió creando los archivos desde la terminal y verificando con `wc -l` antes de cada commit. Otro problema fue deshacer los efectos de una acción: no basta con restaurar la posición de Pedro, porque recoger, usar y soltar cambian el mapa y el inventario. Se resolvió guardando en cada `Accion` el estado de antes y de después, más el objeto afectado con su posición en el inventario.

Por último, se detectó una discrepancia entre el código inicial del juego y la especificación (valores de salud, daño y prioridad de la victoria), que se corrigió alineando el código a la spec.

9. Declaración de uso de inteligencia artificial

Se usó un asistente de inteligencia artificial durante el proyecto. En la Entrega 1 ayudó a redactar el borrador de los artefactos de especificación a partir del documento de diseño, que luego fueron revisados y ajustados. En la entrega final ayudó a escribir el código de `rutas.py`, `juego.py` y `main.py`, las pruebas de `test_juego.py`, los borradores de ANALISIS.md, de la trazabilidad y de este informe.

El código propuesto se comparó contra las especificaciones, se ejecutó con la suite de pruebas y se corrigió donde no coincidía con ellas. El estudiante es responsable de poder explicar cada fragmento en la sustentación, incluyendo el flujo de `_turno`, el funcionamiento de `deshacer` y `rehacer`, y el recorrido de `buscar_ruta`. El detalle está en BITACORA.md.

10. Conclusiones

El proyecto cumple el núcleo jugable: un mapa de 20 x 20 generado por semilla con camino garantizado, un jugador que se mueve por turnos, dos enemigos que lo persiguen con el camino más corto, objetos, inventario, historial con deshacer y rehacer que restaura también los efectos, eventos en orden y condiciones de victoria y derrota, todo desde una consola. Cuatro de las estructuras previstas se implementaron desde cero y están integradas al sistema.

Cada estructura resuelve una necesidad real: las pilas encajan con deshacer porque este siempre actúa sobre lo más reciente; la cola sirve tanto a los eventos como al recorrido en anchura; la lista enlazada conserva el orden del inventario; y el BFS sobre un grafo implícito da el camino más corto sin construir aristas. Las decisiones se justifican con su operación dominante y se comparan con una alternativa descartada.

Quedaron fuera el AVL, el montículo, la cola de turnos propia, el antídoto y el aturdimiento, por lo que las funcionalidades de búsqueda de entidades y ranking (F-08) no están disponibles. Una lección clara es que repartir la implementación de las estructuras a lo largo de las semanas, integrando cada una al sistema apenas se termina, habría permitido llegar a la entrega con el conjunto completo. Como mejoras futuras quedan el AVL de entidades, el montículo de ranking, A* con la comparación de celdas exploradas frente a BFS, y las pruebas de integración pendientes.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Sedgewick, R., y Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley.

Python Software Foundation. (2024). *The Python language reference*. https://docs.python.org/3/reference/

Krekel, H., y colaboradores de pytest. (2024). *pytest documentation*. https://docs.pytest.org/

Universidad Cooperativa de Colombia. (2026). *Proyecto integrador, Temática A: Motor de juego 2D con pathfinding* [Documento técnico del curso Estructuras de Datos, ID 706132].